ADR 0005 — Inference backend (swappable abstraction)

Status: Proposed (Marv to confirm) **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0001, ADR 0002, docs/WRITEUP_INFO.md, src/baymax/service/inference.py, src/baymax/service/api.py

---

Context

The BAYMAX thesis (see docs/WRITEUP_INFO.md) is a head-to-head benchmark between GPT-4, Claude, and a fine-tuned Qwen 2.5 1.5B model on the same reproducible scenario suite. For that comparison to be apples-to-apples:

1. The same agent loop (ADR 0001) and the same tool-call contract (ADR 0002) must run against every backend with no code changes elsewhere
2. The inference layer must be a swappable component — the agent core does not import OpenAI, Anthropic, or MLX directly
3. Adding a new backend (e.g. Llama 3.2 1B, or distilled GPT-4 traces) should be a single class addition with no edits to agent core, eval harness, or telemetry

Backends differ on:

- Authentication shape (API key for OpenAI/Anthropic, model file path for MLX local)
- Native tool-call format (OpenAI function-calling, Anthropic tool-use XML, raw text for some local models)
- Streaming protocol (SSE vs chunked text vs not supported)
- Latency and cost characteristics (network round-trip vs local GPU inference)

A swappable abstraction is therefore a load-bearing piece of infrastructure for the project's central research question.

Decision

Define a `Backend` protocol with one core method, `invoke(messages, tools, ...) → response`. Concrete implementations live in `src/baymax/service/inference.py`:

- `OpenAIBackend` — wraps `openai` SDK; converts internal tool-call format to OpenAI function-call format
- `AnthropicBackend` — wraps `anthropic` SDK; converts to Anthropic tool-use XML
- `MLXBackend` — wraps `mlx-lm`; serves Qwen 2.5 1.5B + LoRA adapter from local model files

Backend selection is driven by an environment variable / config flag at startup. The FastAPI service (`src/baymax/service/api.py`) instantiates one backend per process; the agent core receives it as a constructor dependency and never sees the concrete class.

Authentication per ADR 0002 §6 — credentials read from environment variables only:

- `OPENAI_API_KEY` for OpenAI
- `ANTHROPIC_API_KEY` for Anthropic
- `MLX_MODEL_PATH` + `MLX_ADAPTER_PATH` for local MLX

Each backend is responsible for translating the project's internal tool-call shape to/from its native format. The agent core's view of "what the model said" is normalised before it leaves the inference module.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Documentation depth: brief, honest, readable when future-you re-reads in 6 months. Not aiming for interview-defensible depth (artifact-first project, per 2026-08-24 scope revision).]

- **WHY a protocol / structural interface, not inheritance from a base class**: any class with the right method shape works, no framework dependency, no forced hierarchy for something as thin as "call an LLM and get a response back."
- **WHY not LangChain / an off-the-shelf agent framework abstraction**: LangChain carries assumptions about prompt templates, chain composition, and memory that fight the agent loop locked in ADR 0001; adopting it means either working against the framework or accepting design decisions that conflict with the project's own. Per the CV's "don't let AI own architecture" rule, the abstraction is ours to control.
- **WHY backend translation lives INSIDE the inference module, not in the agent core**: the entire research thesis is comparing backends. Every mention of `openai`, `anthropic`, or `mlx` in the agent core would be a provider leak that undermines the thesis. Translation belongs at the boundary; the agent core sees a normalized shape.
- **WHY env-driven backend selection, not runtime polymorphism or per-request routing**: Ronin's eval harness sweeps backends by spinning up multiple processes with different env vars — no code changes required per backend swap. Runtime polymorphism would add per-request overhead and per-request configuration state for capability the project doesn't need in v1.
- **WHY the protocol has one core method (`invoke` — later split into `plan_task` + `interpret_results` in ADR 0011)**: minimum surface area = maximum swappability. Every additional required method is a translation cost for a new backend author. Keep it thin.
- **WHY this ADR is v1-scoped and superseded for v2 by ADR 0011**: the v1 shape shipped adequately for the OpenAI-only Phase 1 target. Adding MLX + retry policy + warmup + registry integration in Phase 2 demanded a richer surface — hence 0011. Rather than mutate this ADR, 0011 stands as the v2 iteration and this one stays historical.

Alternatives considered

Alternative A — Tight coupling to one provider (e.g. OpenAI only)

Use the OpenAI SDK directly throughout the agent core and the service layer.

why rejected — the entire project thesis is comparing backends. Coupling to one provider would either force us to drop the thesis or rewrite half the codebase to add a second backend later. The cost-of-future-change is too high relative to the cost-of-now-abstraction.

Alternative B — Use LangChain or a similar agent framework's backend abstraction

Adopt LangChain's `BaseLLM` / `ChatModel` hierarchy and use their pre-built backend wrappers.

why rejected — LangChain's abstractions carry assumptions about prompt templates, memory, and chaining that don't match our agent loop (ADR 0001). Pulling them in would either fight the framework or accept its design decisions that conflict with ours. The CV's "don't let AI own architecture" rule applies — we own the abstraction.

Alternative C — Single OpenAI-compatible client (point at different base URLs)

Use a single OpenAI-format client; point it at OpenAI for GPT-4, Anthropic-compat shim for Claude, and a local llama.cpp / vLLM endpoint that exposes the OpenAI format for Qwen.

why rejected — Anthropic's tool-use API differs structurally from OpenAI's function-calling (XML vs JSON, different streaming events); shimming loses fidelity on tool-call edge cases. For Qwen via MLX, exposing an OpenAI-compatible HTTP server adds a process boundary and latency for no benefit in the single-process v1 setup.

Consequences

Positive

- The thesis is testable: any backend can be swapped in with a single env var change
- Each backend's translation logic is isolated; bugs in one don't leak into others
- New backends (Llama 3.2 1B, distilled traces, future Qwen variants) are single-file additions
- Eval harness can run baseline-vs-candidate sweeps by spinning up multiple processes with different env vars

Negative

- 3+ backend implementations to maintain; provider SDK API drift means periodic updates
- Each backend's tool-call translation is its own subtle correctness surface — needs per-backend tests against the same scenario suite
- Latency budgets per stage (ADR 0006) need separate calibration per backend

Neutral

- The `Backend` protocol does not address streaming differences in v1 (deferred per RONIN_SYNC.txt #11); each backend returns the full response and the service layer doesn't expose intermediate tokens

---

Open questions / follow-ups

- **Streaming**: deferred to v2 (DEFERRED.md). When added, the protocol grows a streaming variant.

- **Multi-backend routing within a single request** (e.g., small-model triage → large-model fallback) — deferred indefinitely; would change the protocol shape.
- **Provider-specific failure semantics** (rate limits, content filtering, refusals): normalised to the project's `ErrorType` enum in the backend translation layer; specific mapping documented per backend in its docstring.

References

- OpenAI function-calling: https://platform.openai.com/docs/guides/function-calling
- Anthropic tool use: https://docs.anthropic.com/en/docs/build-with-claude/tool-use
- MLX-LM: https://github.com/ml-explore/mlx-examples/tree/main/llms
- ADR 0002 for the boundary contract
- docs/WRITEUP_INFO.md for the comparison thesis this abstraction serves

---

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.